

BASES TECNOLÓGICAS PARA EL SERVICIO

MÓDULO: PHP

ACTIVIDAD: Reporte de la práctica –
Segundo avance. Práctica Desarrollo de
APIs REST

ALUMNO: CESAR CEDILLO
RODRÍGUEZ

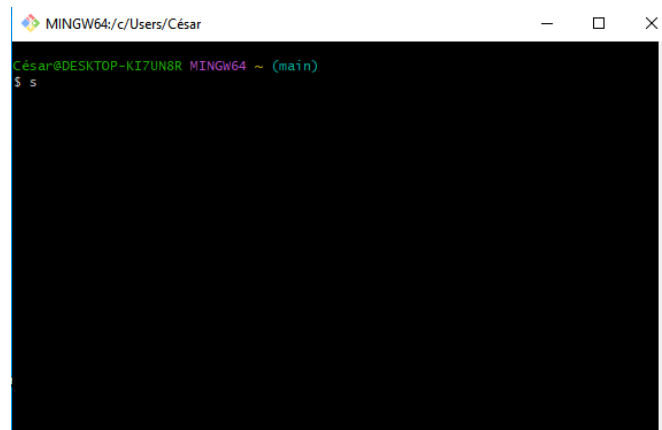
GRUPO: 8

DOCENTE: JUAN MANUEL
TOLENTINO MORALES

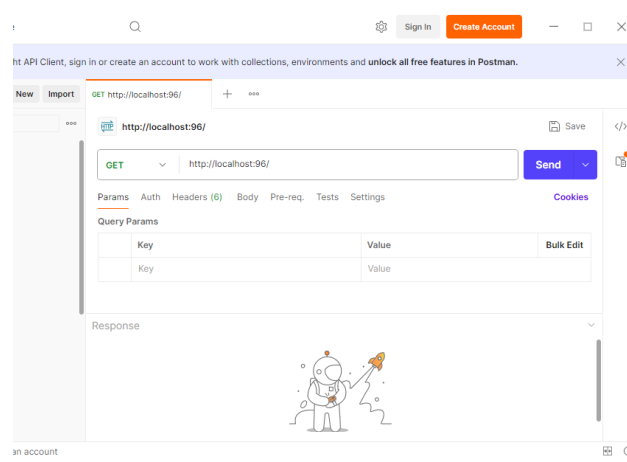
REPORTE DE AVANCES DE LA PRÁCTICA

En este reporte se describe el avance de la práctica de **Desarrollo de APIs REST con Laravel**. Para el desarrollo del proyecto se está utilizando **Docker**, con el fin de facilitar la configuración del entorno y asegurar que sea portable.. Actualmente se ha completado la fase de preparación del entorno de trabajo.

Como primer paso, se verificó que todas las herramientas necesarias estuvieran instaladas correctamente. Entre ellas se encuentran **Git** para el control de versiones, **Docker Desktop** para la gestión de contenedores, **Visual Studio Code** como editor de código y **Postman** para realizar pruebas de la API. Al contar con todas estas herramientas, fue posible avanzar sin mayores inconvenientes.



Consola git



postman

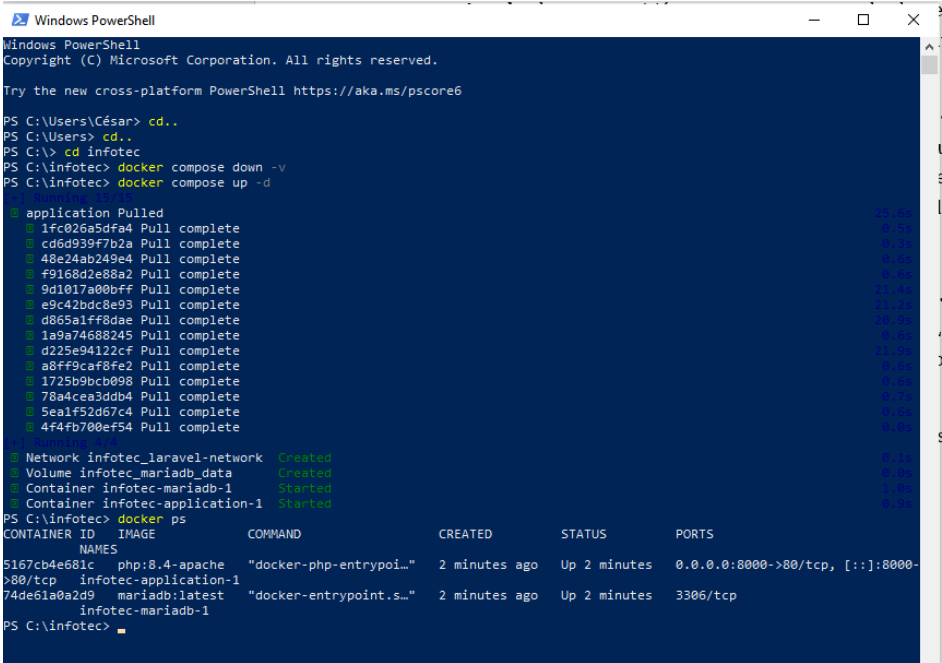
Posteriormente, se creó la estructura inicial del proyecto. Se generó una carpeta principal llamada **infotec**, dentro de la cual se creó la carpeta **src**, destinada a contener todo el código de la aplicación Laravel. Esta estructura es sencilla, pero permite mantener el proyecto ordenado desde el inicio y seguir buenas prácticas de desarrollo.

Una vez creada la estructura, se abrió el proyecto en **Visual Studio Code** directamente desde la carpeta *infotec*. Esto facilitó el acceso a los archivos y permitió trabajar de forma más cómoda, ya que el editor muestra toda la estructura del proyecto sin necesidad de estar navegando constantemente entre carpetas desde la terminal.

El paso más importante en esta etapa fue la creación y configuración del archivo **docker-compose.yml**, el cual define los servicios necesarios para el funcionamiento del proyecto. Inicialmente, se intentó utilizar la imagen de **Bitnami para Laravel**, sin embargo, al levantar los contenedores se presentó el error `chown: invalid group: 'bitnami:bitnami'`, relacionado con problemas de permisos dentro del contenedor. Este error impedía que la aplicación se ejecutara correctamente.

Debido a este inconveniente, fue necesario **modificar el archivo YAML** y cambiar la imagen utilizada para la aplicación. Como solución, se optó por utilizar una **imagen estándar de PHP 8.4 con Apache**, lo que permitió tener un mayor control sobre el entorno y evitar los problemas de permisos. A partir de este cambio, la instalación de Laravel y sus dependencias se realizó directamente dentro del contenedor.

De este modo, la fase de preparación del entorno se completó de manera satisfactoria. A pesar del problema inicial con la imagen de Bitnami, se logró una solución funcional y estable utilizando una imagen estándar de PHP. El proyecto cuenta con una estructura clara, una base de datos configurada y un entorno listo para comenzar con el desarrollo de la API.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\César> cd..
PS C:\Users> cd..
PS C:\> cd infotec
PS C:\infotec> docker compose down -v
PS C:\infotec> docker compose up -d

application Pulled
  1fc026a5dfa4 Pull complete
  cd6d939f7b2a Pull complete
  48e24ab249e4 Pull complete
  f9168de288e2 Pull complete
  9d1017a00bff Pull complete
  e9c42bdc8e93 Pull complete
  d865a1ff8dae Pull complete
  1a9a74688245 Pull complete
  d225e94122cf Pull complete
  a8ff9caf8fe2 Pull complete
  1725b9bcb098 Pull complete
  78a4cea3ddb4 Pull complete
  5ealf52d67c4 Pull complete
  4f4fb700ef54 Pull complete

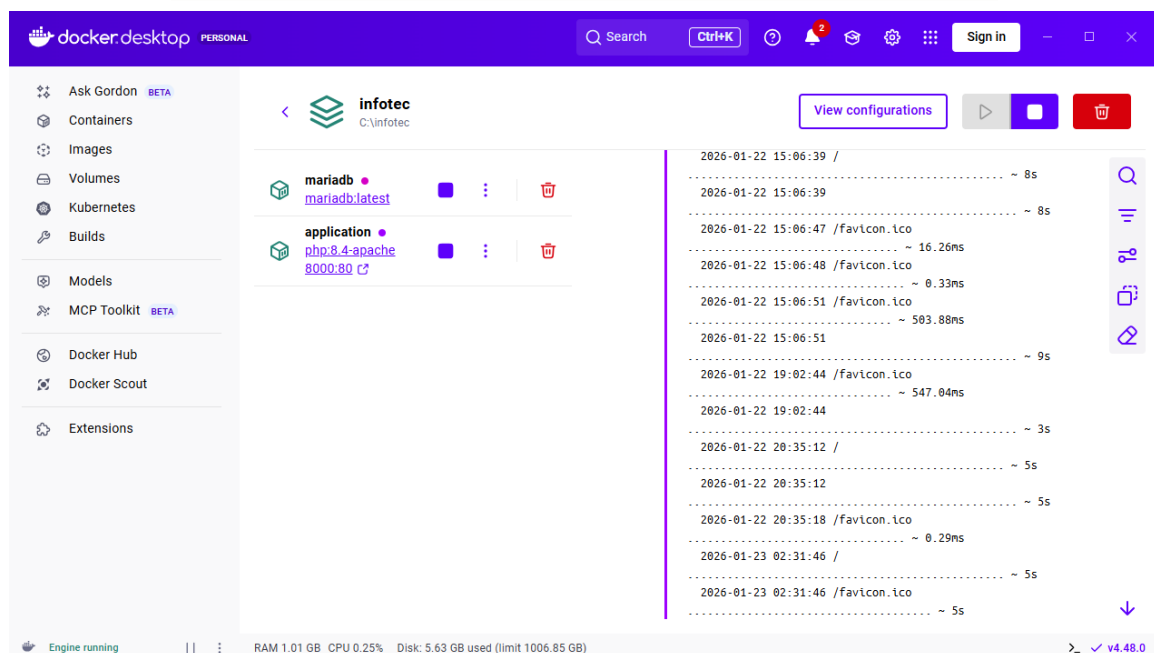
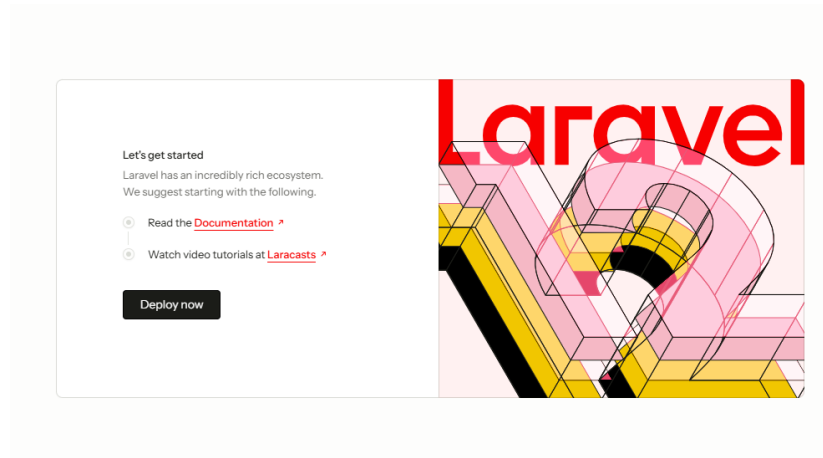
Network infotec_laravel-network Created
Volume infotec_mariadb_data Created
Container infotec-mariadb-1 Started
Container infotec-application-1 Started

PS C:\infotec> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
5167cb4e681c   php:8.4-apache                      "docker-php-entrypoi..." 2 minutes ago  Up 2 minutes  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
74de61a0a2d9   infotec-application-1              "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  3306/tcp
74de61a0a2d9   infotec-mariadb-1                  "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  3306/tcp
```

Como siguientes pasos, se planea levantar los contenedores, verificar que la aplicación funcione correctamente y comenzar con el desarrollo de la API en Laravel. Esto incluirá la creación de

modelos, controladores, rutas, migraciones de base de datos y pruebas para asegurar que las operaciones básicas funcionen correctamente.

Por el momento, el proyecto avanza de forma adecuada y este reporte se actualizará conforme continúe la práctica.



ETAPA 2

En esta etapa se tiene como objetivo diseñar e implementar la estructura base de una API REST en Laravel para la gestión de eventos, utilizando controladores API, modelos, migraciones y rutas, preparando el proyecto para su posterior prueba mediante Postman.

Verificación del uso de Artisan

Se verificó que el entorno Laravel permite la ejecución de comandos Artisan desde el contenedor Docker, lo cual confirma que el framework se encuentra correctamente instalado y funcional.

```
PS C:\infotec> docker compose exec application php artisan --version
Laravel Framework 12.48.1
PS C:\infotec>
```

2.2 Configuración de la base de datos

Se configuró Laravel para utilizar el servicio de base de datos MariaDB definido en Docker, actualizando las variables de conexión en el archivo .env y limpiando la caché de configuración para aplicar los cambios.

```
DB_CONNECTION=mysql
DB_HOST=mariadb
DB_PORT=3306
DB_DATABASE=bitnami_myapp
DB_USERNAME=bn_myapp
DB_PASSWORD=bitnam1_my@pp
```

2.3 Creación del modelo, controlador y migración

Se generó el modelo Evento junto con su controlador API y la migración correspondiente mediante el uso de Artisan, lo que permitió establecer la base de la entidad principal del sistema.

2.5 Configuración del modelo Evento

Se configuró el modelo Evento para permitir la asignación masiva de los campos definidos en la migración y se especificó la tabla asociada en la base de datos.

```
1  <?php
2  namespace App\Models;
3  use Illuminate\Database\Eloquent\Model;
4  class Evento extends Model
5  {
6      protected $table = 'eventos';
7      protected $fillable = [
8          'titulo',
9          'descripcion',
10         'fecha_inicio',
11         'fecha_fin',
12         'ubicacion'
13     ];
14 }
```

2.7 Configuración de rutas de la API

Se creó y configuró el archivo routes/api.php, donde se definieron las rutas REST que exponen los endpoints para la gestión de eventos mediante el prefijo /api.

2.8 Verificación de las rutas

Se verificó que las rutas de la API se encuentren correctamente registradas en Laravel mediante el listado de rutas.

```
C:\infotec>
```

```
GET|HEAD /
.....
.....
.....

GET|HEAD storage/{path}
.....
.....
..... storage.local

GET|HEAD up ....
```

```
PS C:\infotec> docker compose exec application php artisan --version
Laravel Framework 12.48.1
PS C:\infotec>
PS C:\infotec> docker compose exec application ls database/migration
ls: cannot access 'database/migration': No such file or directory
PS C:\infotec> docker compose exec application pwd
/var/www/html
PS C:\infotec> docker compose exec application ls
README.md  artisan  composer.json  config  licenses  phpunit.xml  resources  storage  vendor
app        bootstrap  composer.lock  database  package.json  public  routes  tests  vite.config.js
PS C:\infotec> docker compose exec application ls database/migrations
0001_01_01_000000_create_users_table.php  0001_01_01_000002_create_jobs_table.php
0001_01_01_000001_create_cache_table.php  2026_01_29_193948_create_eventos_table.php
PS C:\infotec> docker compose exec application php artisan migrate

INFO: Preparing database.

Creating migration table ..... 477.54ms DONE

INFO: Running migrations.

0001_01_01_000000_create_users_table ..... 693.36ms DONE
0001_01_01_000001_create_cache_table ..... 34.99ms DONE
0001_01_01_000002_create_jobs_table ..... 369.17ms DONE
2026_01_29_193948_create_eventos_table ..... 17.89ms DONE

PS C:\infotec> docker compose exec application ls routes
console.php  web.php
PS C:\infotec> docker compose exec application touch routes/api.php
PS C:\infotec> docker compose exec application ls routes
api.php  console.php  web.php
```

RESUMEN DE LA ETAPA 2

En esta etapa se desarrolló la estructura base de una API REST utilizando Laravel, orientada a la gestión de eventos. Se crearon el modelo, la migración y el controlador API correspondientes a la entidad *Evento*, definiendo la estructura de la base de datos y la lógica básica para las operaciones CRUD. Asimismo, se configuraron las rutas de la API en el archivo `api.php`, quedando registrados los endpoints necesarios para el acceso a los recursos mediante solicitudes HTTP y dejando la aplicación preparada para su validación en la siguiente etapa.

3. Pruebas de la API REST mediante Postman

Una vez implementada la API REST para la gestión de eventos, se procedió a realizar las pruebas de funcionamiento utilizando la herramienta Postman, con el objetivo de verificar la correcta operación de los distintos endpoints definidos y validar el intercambio de información en formato JSON.

Las pruebas se realizaron accediendo al servicio a través de la URL base:

<http://localhost:8000/api>

A continuación, se describen las pruebas efectuadas para cada una de las operaciones CRUD correspondientes a la entidad Evento.

3.1 Consulta de todos los eventos (GET)

Se realizó una solicitud de tipo GET al endpoint /eventos, con el fin de recuperar la lista completa de eventos almacenados en la base de datos.

Resultado:

El servidor respondió con un arreglo de objetos en formato JSON, junto con un código de estado 200 (OK), confirmando que la consulta se ejecutó correctamente.

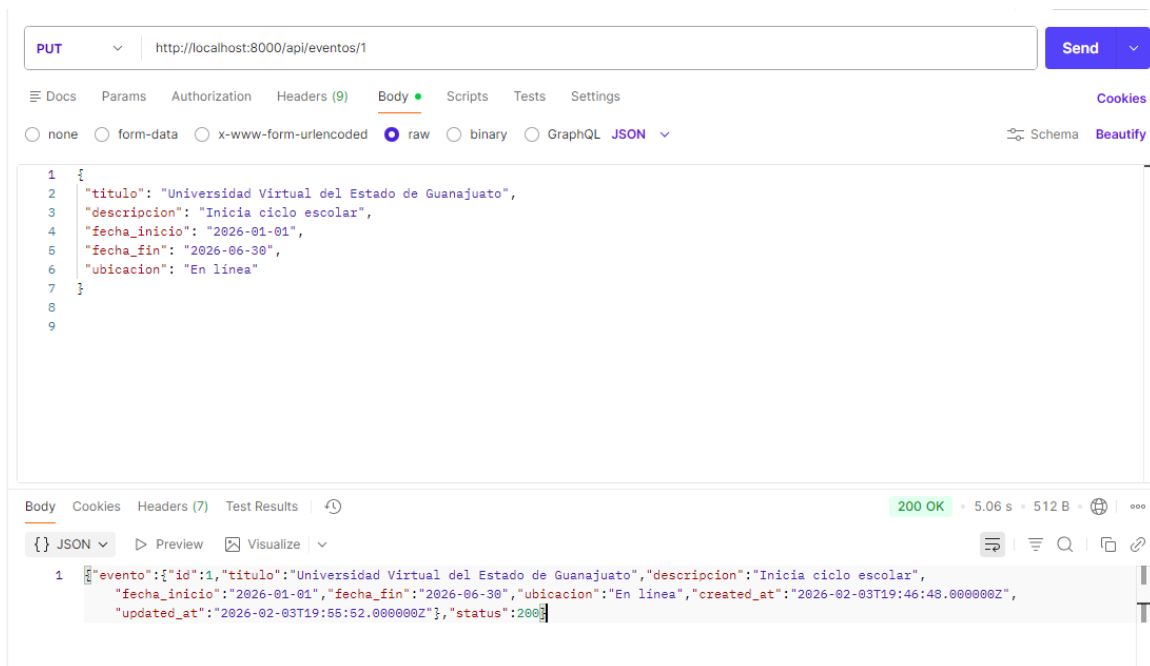


3.2 Consulta de un evento específico (GET por ID)

Se efectuó una solicitud GET al endpoint /eventos/{id}, proporcionando el identificador de un evento existente.

Resultado:

La API devolvió la información correspondiente al evento solicitado en formato JSON y un código de estado 200 (OK).



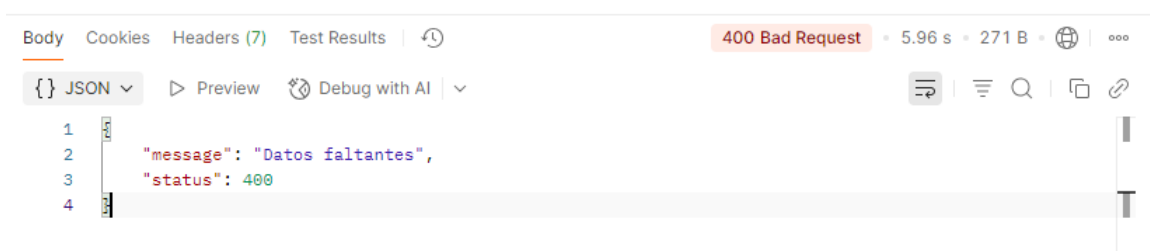
En caso de que el identificador no existiera, la API respondió con un mensaje de error y un código 404 (Not Found), validando el manejo adecuado de errores.

3.3 Registro de un nuevo evento (POST)

Para la creación de un nuevo evento, se envió una solicitud POST al endpoint /eventos, incluyendo en el cuerpo de la petición los datos requeridos (título, descripción, fechas y ubicación).

Resultado:

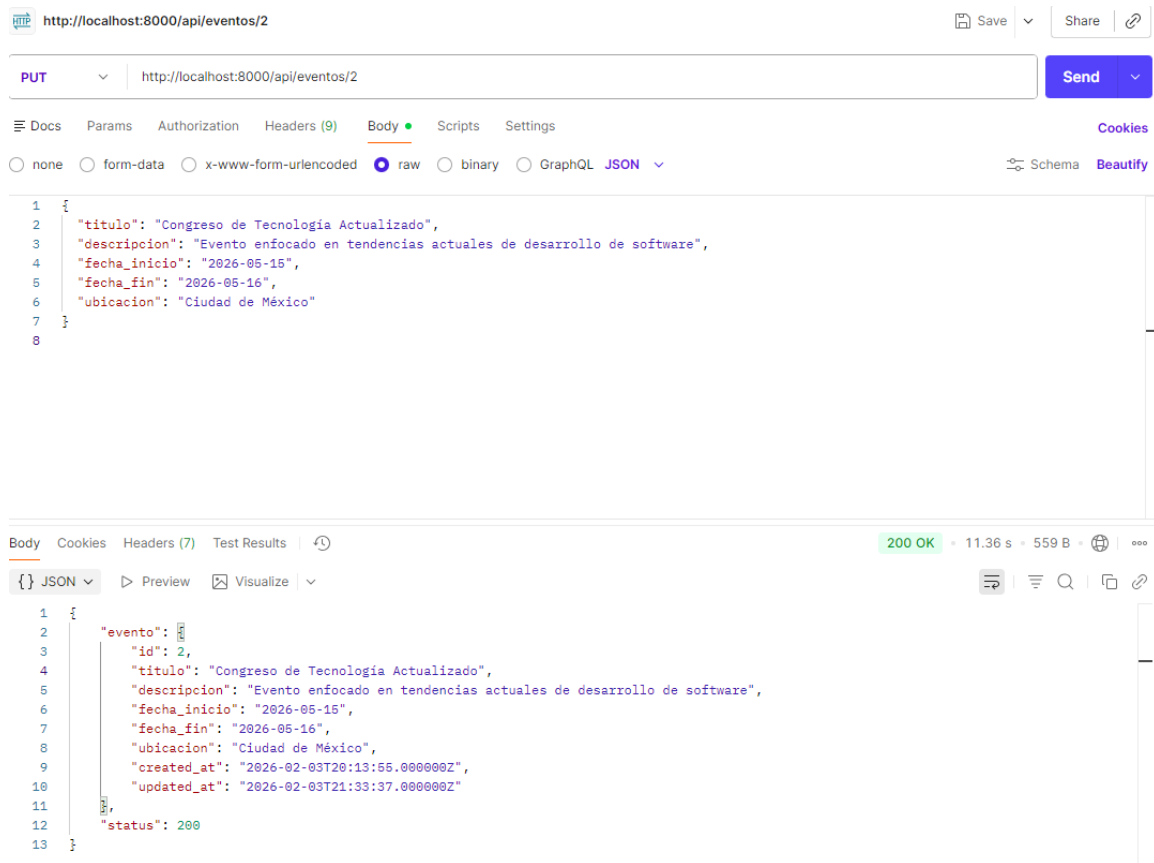
La API registró correctamente el evento en la base de datos y respondió con el objeto creado en formato JSON y un código de estado 201 (Created), confirmando la correcta inserción de datos.



Si no se introducen los datos completos el resultado es un mensaje de datos faltantes.

3.4 Actualización de un evento (PUT)

Se realizó una solicitud PUT al endpoint `/eventos/{id}`, enviando los nuevos datos del evento a actualizar.



Resultado:

El evento fue actualizado correctamente y la API devolvió el registro modificado junto con un código de estado 200 (OK).

Cuando se intentó actualizar un evento inexistente, la API respondió con un código 404 (Not Found).

HTTP `http://localhost:8000/api/eventos/1` Save Share

PUT `http://localhost:8000/api/eventos/1` Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "titulo": "Congreso de Tecnología",
3   "descripcion": "Evento enfocado en tendencias actuales de la tecnología",
4   "fecha_inicio": "2026-05-15",
5   "fecha_fin": "2026-05-16",
6   "ubicacion": "Ciudad de México"
7 }
8
```

Body Cookies Headers (7) Test Results 404 Not Found • 1.88 s • 274 B

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Evento no encontrado",
3   "status": 404
4 }
```

3.5 Eliminación de un evento (DELETE)

Finalmente, se ejecutó una solicitud DELETE al endpoint `/eventos/{id}` para eliminar un evento específico.

GET Get data DEL `http://localhost:8000/api/eventos/1` No environment

HTTP `http://localhost:8000/api/eventos/1` Save Share

DELETE `http://localhost:8000/api/eventos/1` Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

Pre-request 1 Use JavaScript to write tests, visualize response, and more. Ctrl+Alt+P to Ask AI

Post-response

Body Cookies Headers (7) Test Results 200 OK • 1.78 s • 263 B

{ } JSON Preview Visualize

```
1 {
2   "message": "Evento eliminado",
3   "status": 200
4 }
```

Resultado:

La API confirmó la eliminación del registro mediante un mensaje en formato JSON y un código de estado 200 (OK).

En caso de que el evento no existiera, se devolvió un mensaje de error correspondiente.

3.6 Consideración de entidades Ponentes y Asistentes

De acuerdo con el planteamiento general del guion de la práctica, la API REST desarrollada contempla la gestión de eventos, ponentes y asistentes. En esta etapa, se implementó y probó completamente la entidad Evento, la cual funciona como base para la arquitectura de la API.

Asimismo, se realizó el análisis del diseño necesario para la integración de las entidades Ponentes y Asistentes, considerando que estas seguirán el mismo patrón de desarrollo aplicado a la entidad Evento, utilizando controladores REST, rutas API y operaciones CRUD. Esta preparación permite que la API sea escalable y facilita la incorporación de dichas entidades en etapas posteriores sin modificar la estructura principal del sistema.

Durante el desarrollo de la **Parte 3 de la práctica**, se realizaron las modificaciones necesarias al API REST para gestionar las entidades **Ponentes** y **Asistentes**, siguiendo el patrón **Modelo-Vista-Controlador (MVC)** y utilizando Laravel como framework de desarrollo.

```
<?php
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
// Agregar el controlador EventoController
use App\Http\Controllers\EventoController;

/**
 * Rutas para el recurso Evento.
 */
// Recuperar todos los eventos
Route::get('/eventos', [EventoController::class, 'index']);
// Almacenar un evento nuevo
Route::post('/eventos', [EventoController::class, 'store']);
// Recuperar un evento específico
Route::get('/eventos/{id}', [EventoController::class, 'show']);
// Actualizar un evento específico
Route::put('/eventos/{id}', [EventoController::class, 'update']);
// Eliminar un evento específico
Route::delete('/eventos/{id}', [EventoController::class, 'destroy']);

// Ponentes
use App\Http\Controllers\PonenteController;
Route::get('/ponentes', [PonenteController::class, 'index']);
Route::post('/ponentes', [PonenteController::class, 'store']);
Route::get('/ponentes/{id}', [PonenteController::class, 'show']);
Route::put('/ponentes/{id}', [PonenteController::class, 'update']);
Route::delete('/ponentes/{id}', [PonenteController::class, 'destroy']);

// Asistentes
use App\Http\Controllers\AsistenteController;
Route::get('/asistentes', [AsistenteController::class, 'index']);
Route::post('/asistentes', [AsistenteController::class, 'store']);
Route::get('/asistentes/{id}', [AsistenteController::class, 'show']);
Route::put('/asistentes/{id}', [AsistenteController::class, 'update']);
Route::delete('/asistentes/{id}', [AsistenteController::class, 'destroy']);

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');
```

Ctrl+I for Command, Ctrl+L for Agent

Se implementaron los **modelos**, **controladores** y **rutas** correspondientes para cada entidad, permitiendo la correcta interacción con la base de datos mediante Eloquent ORM. Asimismo, se crearon las **migraciones** necesarias para definir la estructura de las tablas y sus relaciones, garantizando la integridad de la información almacenada.

CONTROLADORES

Ponente controller

```
<?php

namespace App\Http\Controllers;

use App\Models\Ponente;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class PonenteController extends Controller
{
    public function index()
    {
        $ponentes = Ponente::all();
        return response()->json([
            'ponentes' => $ponentes,
            'status' => 200,
        ]);
    }

    public function store(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'nombre' => 'required|string|max:255',
            'biografia' => 'required|string',
        ]);

        if ($validator->fails()) {
            return response()->json([
                'message' => 'Datos faltantes o inválidos',
                'errors' => $validator->errors(),
                'status' => 400,
            ], 400);
        }

        $ponente = Ponente::create([
            'nombre' => $request->nombre,
            'biografia' => $request->biografia,
        ]);

        if (!$ponente) {
            return response()->json([
                'message' => 'Error al crear el ponente',
                'status' => 500,
            ], 500);
        }

        return response()->json([
            'ponente' => $ponente,
            'status' => 201,
        ], 201);
    }

    public function show($id)
    {
        $ponente = Ponente::find($id);
```

```

        if (!$ponente) {
            return response()->json([
                'message' => 'Ponente no encontrado',
                'status' => 404,
            ], 404);
        }

        return response()->json([
            'ponente' => $ponente,
            'status' => 200,
        ]);
    }

    public function update(Request $request, $id)
    {
        $ponente = Ponente::find($id);

        if (!$ponente) {
            return response()->json([
                'message' => 'Ponente no encontrado',
                'status' => 404,
            ], 404);
        }

        $validator = Validator::make($request->all(), [
            'nombre' => 'required|string|max:255',
            'biografia' => 'required|string',
        ]);

        if ($validator->fails()) {
            return response()->json([
                'message' => 'Datos faltantes o inválidos',
                'errors' => $validator->errors(),
                'status' => 400,
            ], 400);
        }

        $ponente->nombre = $request->nombre;
        $ponente->biografia = $request->biografia;
        $ponente->save();

        return response()->json([
            'ponente' => $ponente,
            'status' => 200,
        ]);
    }

    public function destroy($id)
    {
        $ponente = Ponente::find($id);

        if (!$ponente) {
            return response()->json([
                'message' => 'Ponente no encontrado',
                'status' => 404,
            ], 404);
        }

        $ponente->delete();

        return response()->json([
            'message' => 'Ponente eliminado',
            'status' => 200,
        ]);
    }
}

```

Asistente controller

```
<?php

namespace App\Http\Controllers;

use App\Models\Asistente;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class AsistenteController extends Controller
{
    public function index()
    {
        $asistentes = Asistente::all();
        return response()->json([
            'asistentes' => $asistentes,
            'status' => 200,
        ]);
    }

    public function store(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'nombre' => 'required|string|max:255',
            'email' => 'required|email|max:255',
            'telefono' => 'required|string|max:20',
            'evento_id' => 'required|exists:eventos,id',
        ]);

        if ($validator->fails()) {
            return response()->json([
                'message' => 'Datos faltantes o inválidos',
                'errors' => $validator->errors(),
                'status' => 400,
            ], 400);
        }

        $asistente = Asistente::create([
            'nombre' => $request->nombre,
            'email' => $request->email,
            'telefono' => $request->telefono,
            'evento_id' => $request->evento_id,
        ]);

        if (!$asistente) {
            return response()->json([
                'message' => 'Error al crear el asistente',
                'status' => 500,
            ], 500);
        }

        return response()->json([
            'asistente' => $asistente,
            'status' => 201,
        ], 201);
    }

    public function show($id)
    {
        $asistente = Asistente::find($id);

        if (!$asistente) {
            return response()->json([
                'message' => 'Asistente no encontrado',
                'status' => 404,
            ], 404);
        }
    }
}
```

```

        return response()->json([
            'asistente' => $asistente,
            'status' => 200,
        ]);
    }

    public function update(Request $request, $id)
    {
        $asistente = Asistente::find($id);

        if (!$asistente) {
            return response()->json([
                'message' => 'Asistente no encontrado',
                'status' => 404,
            ], 404);
        }

        $validator = Validator::make($request->all(), [
            'nombre' => 'required|string|max:255',
            'email' => 'required|email|max:255',
            'telefono' => 'required|string|max:20',
            'evento_id' => 'required|exists:eventos,id',
        ]);

        if ($validator->fails()) {
            return response()->json([
                'message' => 'Datos faltantes o inválidos',
                'errors' => $validator->errors(),
                'status' => 400,
            ], 400);
        }

        $asistente->nombre = $request->nombre;
        $asistente->email = $request->email;
        $asistente->telefono = $request->telefono;
        $asistente->evento_id = $request->evento_id;
        $asistente->save();

        return response()->json([
            'asistente' => $asistente,
            'status' => 200,
        ]);
    }

    public function destroy($id)
    {
        $asistente = Asistente::find($id);

        if (!$asistente) {
            return response()->json([
                'message' => 'Asistente no encontrado',
                'status' => 404,
            ], 404);
        }

        $asistente->delete();

        return response()->json([
            'message' => 'Asistente eliminado',
            'status' => 200,
        ]);
    }
}

```

Asistente.php

```
src > app > Models > Asistente.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Asistente extends Model
8  {
9      protected $table = 'asistentes';
10
11      protected $fillable = [
12          'nombre',
13          'email',
14          'telefono',
15          'evento_id'
16      ];
17  }
18
```

Edit Ctrl+I Chat Ctrl+L ...

Ponente.php

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Ponente extends Model
8  {
9      protected $table = 'ponentes';
10
11      protected $fillable = [
12          'nombre',
13          'biografia'
14      ];
15  }
16
```

Edit Ctrl+I Chat Ctrl+L ...

Migrations asistentes

```

src > database > migrations > 2025_01_01_000002_create_asistentes_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('asistentes', function (Blueprint $table) {
15             $table->id();
16             $table->string('nombre');
17             $table->string('email');
18             $table->string('telefono');
19             $table->unsignedBigInteger('evento_id');
20             $table->foreign('evento_id')->references('id')->on('eventos')->onDelete('cascade');
21             $table->timestamps();
22         });
23     }
24
25     /**
26      * Reverse the migrations.
27      */
28     public function down(): void
29     {
30         Schema::dropIfExists('asistentes');
31     }
32 };
33

```

Migrations ponentes

```

src > database > migrations > 2025_01_01_000001_create_ponentes_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('ponentes', function (Blueprint $table) {
15             $table->id();
16             $table->string('nombre');
17             $table->text('biografia');
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      */
25     public function down(): void
26     {
27         Schema::dropIfExists('ponentes');
28     }
29 };
30

```

Para comprobar el correcto funcionamiento del API, se realizaron pruebas utilizando la herramienta **Postman**, validando los métodos **GET** y **POST** para las entidades Ponentes y Asistentes. Las capturas de pantalla anexadas muestran la correcta recepción de solicitudes, así

como las respuestas en formato JSON, confirmando que los datos son almacenados y recuperados de manera adecuada.

Con estas pruebas se verifica que el API cumple con los principios de una arquitectura **REST**, haciendo uso de los métodos HTTP apropiados y separando correctamente la lógica de negocio a través del uso del patrón MVC.

The image displays two screenshots of a REST client interface, likely Postman, showing API requests and their corresponding JSON responses.

First Screenshot: POST request to http://localhost:8000/api/eventos

The request body is a JSON object:

```
1 {
2   "titulo": "Escuela Pública de Código",
3   "descripcion": "Inicia oficialmente el curso de Bases tecnológicas en el servicio público",
4   "fecha_inicio": "2025-01-01",
5   "fecha_fin": "2025-06-30",
6   "ubicacion": "En línea"
7 }
8
9
```

The response is a JSON object with status 201 Created:

```
1 {
2   "evento": {
3     "titulo": "Escuela Pública de Código",
4     "descripcion": "Inicia oficialmente el curso de Bases tecnológicas en el servicio público",
5     "fecha_inicio": "2025-01-01",
6     "fecha_fin": "2025-06-30",
7     "ubicacion": "En línea",
8     "updated_at": "2026-02-03T19:46:49.000000Z",
9     "created_at": "2026-02-03T19:46:49.000000Z",
10    "id": 1
11  },
12  "status": 201
13 }
```

Second Screenshot: POST request to http://localhost:8000/api/ponentes

The request body is a JSON object:

```
1 {
2   "nombre": "Juan Pérez",
3   "biografia": "Experto en Laravel y APIs REST."
4 }
```

The response is a JSON object with status 201 Created:

```
1 {
2   "ponente": {
3     "nombre": "Juan Pérez",
4     "biografia": "Experto en Laravel y APIs REST.",
5     "updated_at": "2026-02-03T20:10:12.000000Z",
6     "created_at": "2026-02-03T20:10:12.000000Z",
7     "id": 1,
8     "status": 201
9   }
10 }
```

http://localhost:8000/api/eventos

SaveShare

POSThttp://localhost:8000/api/eventos

Send

DocsParamsAuthorizationHeaders (9)BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQL JSON

SchemaBeautify

```
1 {
2   "titulo": "Conferencia de Seguridad Informática 2026",
3   "descripcion": "Un encuentro con expertos para discutir las últimas tendencias en ciberseguridad y protección de datos.",
4   "fecha_inicio": "2026-04-10",
5   "fecha_fin": "2026-04-12",
6   "ubicacion": "Auditorio Principal - Torre Infotek"
7 }
```

BodyCookiesHeaders (7)Test Results

201 Created4.78 s634 B

{ } JSONPreviewVisualize

```
1 {
2   "evento": {
3     "titulo": "Conferencia de Seguridad Informática 2026",
4     "descripcion": "Un encuentro con expertos para discutir las últimas tendencias en ciberseguridad y protección de datos.",
5     "fecha_inicio": "2026-04-10",
6     "fecha_fin": "2026-04-12",
7     "ubicacion": "Auditorio Principal - Torre Infotek",
8     "updated_at": "2026-02-03T20:13:55.000000Z",
9     "created_at": "2026-02-03T20:13:55.000000Z",
10    "id": 2
11  },
12  "status": 201
13 }
```

 http://localhost:8000/api/eventos

Save Share

GET http://localhost:8000/api/eventos Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies Headers (7) Test Results 200 OK 4.77 s 632 B

JSON Preview Visualize

```
1 {
2   "eventos": [
3     {
4       "id": 2,
5       "titulo": "Conferencia de Seguridad Informática 2026",
6       "descripcion": "Un encuentro con expertos para discutir las últimas tendencias en ciberseguridad y protección de datos.",
7       "fecha_inicio": "2026-04-10",
8       "fecha_fin": "2026-04-12",
9       "ubicacion": "Auditorio Principal - Torre Infotek",
10      "created_at": "2026-02-03T20:13:55.000000Z",
11      "updated_at": "2026-02-03T20:13:55.000000Z"
12    }
13  ],
14  "status": 200
15 }
```

 http://localhost:8000/api/asistentes

Save Share

GET http://localhost:8000/api/asistentes Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "nombre": "Maria García",
3   "email": "maria@example.com",
4   "telefono": "555-1234",
5   "evento_id":
6 }
```

Body Cookies Headers (7) Test Results 200 OK 1.78 s 250 B

JSON Preview Visualize

```
1 {
2   "asistentes": [],
3   "status": 200
4 }
```